ASB 1305 – Databases in Business & Finance

Assignment 1 — Complete SQL Solutions & Explanations

All 23 Queries · Step-by-Step Explanations · Sample Outputs

How to Use This Guide

Each query is presented in four parts: (1) the SQL code you type in R using sqldf, (2) a plain-English explanation of every keyword used, (3) a sample output table, and (4) any important notes. Read the explanation carefully — it teaches you the *why*, not just the *what*.

TASK 1 — Create the Data Tables in R

R Code to Build All Three Tables

Before writing any SQL we must create the data frames in R. Think of a data frame as a spreadsheet table — rows are observations and columns are variables.

```
library(sqldf) # loads the sqldf package — lets us run SQL on R data frames

# ■■ TABLE 1: firms
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
firms <- data.frame(
firm_id = c(101,102,103,104,105,106,107,108),
firm_name = c('Alpha plc','Beta Ltd','Cobalt SA','Delta Inc',
'Epsilon Co','Fjord GmbH','Gamma Energy','Helios Tech'),
sector = c('Tech','Manufacturing','Energy','Retail',
'Tech','Energy','Energy','Tech'),
country = c('UK','UK','DE','FR','UK','DE','UK','US'),
headquarters = c('London','Manchester','Berlin','Paris',
'Leeds','Frankfurt','Aberdeen','New York'),
founded_year = c(2001,1998,2005,2010,2015,2003,2008,2012)
)

# ■■ TABLE 2: fundamentals
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
fundamentals <- data.frame(
firm_id = c(101,101,102,102,103,103,104,104,105,105,106,106,107,107,108,108),
fiscal_year =
c(2023,2024,2023,2024,2023,2024,2023,2024,2023,2024,2023,2024,2023,2024,2023,2024),
sales_m = c(140,160,190,210,NA,320,80,95,70,NA,260,280,310,330,150,170),
assets_m = c(520,540,820,850,1520,1550,400,420,310,330,1400,1450,1600,1680,600,640),
debt_m = c(190,200,410,430,920,950,220,230,95,110,780,800,980,1000,240,250),
net_income_m = c(22,24,10,14,18,25,5,7,9,NA,16,19,21,24,12,13),
cash_m = c(65,70,45,50,115,120,25,30,58,60,90,95,130,140,40,42)
)

# ■■ TABLE 3: market
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
market <- data.frame(
firm_id = c(101,102,103,104,105,106,107,108),
analyst_count = c(12,9,15,6,5,11,14,8),
esg_score = c(74,56,50,64,82,47,68,79),
market_cap_m = c(1000,1500,2000,800,600,1800,2200,1200),
exchange = c('LSE','LSE','FWB','EPA','LSE','FWB','LSE','NASDAQ')
)
```

Understanding the R data.frame() function

library(sqldf) — loads the sqldf package so R understands SQL commands.

data.frame() — creates a table. Inside the brackets, each *name = c(...)* defines one column. c() means 'combine these values into a vector'.

NA — R's way of writing a missing value, equivalent to NULL in SQL.

firms$firm_id links to fundamentals$firm_id and market$firm_id — these are the foreign keys that let us JOIN the tables together later.

TASK 2 PART A — One-Table Queries (Using 'firms')

Query 1: Retrieve all records from the firms table

```
sqldf('SELECT * FROM firms')
```

Keywords Used: SELECT * FROM

SELECT — tells SQL which columns to return.

***** (asterisk) — a wildcard meaning 'all columns'. Instead of listing every column name, * grabs them all.

FROM firms — tells SQL which table to look in.

This is the most basic SQL query possible. Every query starts with SELECT … FROM.

Sample Output:

firm_id	firm_name	sector	country	headquarters	founded_year
101	Alpha plc	Tech	UK	London	2001
102	Beta Ltd	Manufacturing	UK	Manchester	1998
...	...	...	...	...	...

Query 2: Return only firm_name, sector, and country

```
sqldf('SELECT firm_name, sector, country FROM firms')
```

Keywords Used: SELECT (specific columns)

Instead of *, we list exactly the columns we want separated by commas.

This is called **column projection** — we 'project' only the columns we care about.

SQL reads: *From the firms table, give me only the firm_name, sector, and country columns.*

Sample Output:

firm_name	sector	country
Alpha plc	Tech	UK
Beta Ltd	Manufacturing	UK
Cobalt SA	Energy	DE
...	...	...

Query 3: List all distinct sector values

```
sqldf('SELECT DISTINCT sector FROM firms')
```

Keywords Used: SELECT DISTINCT

DISTINCT removes duplicate rows from the result.

Without DISTINCT, sector 'Tech' would appear 3 times (for Alpha, Epsilon, Helios). DISTINCT keeps only one copy of each unique value.

Useful when you want to know *what categories exist* in a column.

Sample Output:

sector
Tech
Manufacturing
Energy
Retail

Query 4: Find firms located in the UK

```
sqldf("SELECT * FROM firms WHERE country = 'UK'")
```

Keywords Used: WHERE (equality filter)

WHERE is a filter — it keeps only rows that match a condition.

country = 'UK' — the condition. In SQL, text values (strings) are always wrapped in single quotes. Numbers are not.

SQL reads: *Give me all columns from firms, but only for rows where the country column equals UK.*

Sample Output:

firm_id	firm_name	sector	country
101	Alpha plc	Tech	UK
102	Beta Ltd	Manufacturing	UK
105	Epsilon Co	Tech	UK
107	Gamma Energy	Energy	UK

Query 5: Find firms whose names start with 'G' using LIKE

```
sqldf("SELECT * FROM firms WHERE firm_name LIKE 'G%'")
```

Keywords Used: WHERE LIKE % (wildcard)

LIKE is used for pattern matching on text — it checks if a string looks like a pattern.

% (percent sign) is a wildcard meaning 'any characters, any length'.

'G%' means: starts with G, followed by anything.

Other patterns: '%Tech' means ends with Tech; '%Tech%' means contains Tech anywhere.

Sample Output:

firm_id	firm_name	sector
107	Gamma Energy	Energy

Query 6: Find firms whose names contain 'Tech' using LIKE

```
sqldf("SELECT * FROM firms WHERE firm_name LIKE '%Tech%'")
```

Keywords Used: LIKE with % on both sides

'%Tech%' — percent on both sides means 'Tech can appear anywhere in the name'.

This finds: 'Helios **Tech**' (Tech at the end). Note: 'Tech' sector firms are different from firms whose *name* contains Tech — SQL checks exactly the column you specify.

Sample Output:

firm_id	firm_name	sector
108	Helios Tech	Tech

Query 7: Firms founded after 2005 and not in Germany

```
sqldf("SELECT * FROM firms WHERE founded_year > 2005 AND country != 'DE'")
```

Keywords Used: WHERE AND > !=

AND — combines two conditions; BOTH must be true for a row to be included.

> — greater than. founded_year > 2005 means 2006, 2007, 2008… etc.

!= — not equal to. You may also see **<>** which means the same thing.

Rows kept: Delta Inc (2010, FR), Epsilon Co (2015, UK), Gamma Energy (2008, UK), Helios Tech (2012, US).

Sample Output:

firm_id	firm_name	country	founded_year
104	Delta Inc	FR	2010
105	Epsilon Co	UK	2015
107	Gamma Energy	UK	2008
108	Helios Tech	US	2012

One-Table Queries Using 'fundamentals'

Query 8: Return firm_id, fiscal_year, sales_m, net_income_m where sales_m > 200

```
sqldf('SELECT firm_id, fiscal_year, sales_m, net_income_m
FROM fundamentals
WHERE sales_m > 200')
```

Keywords Used: SELECT (columns) FROM WHERE >

We combine column selection with a numeric filter here.

sales_m > 200 — only rows where sales exceed 200 million.

Notice: rows with sales_m = NA (NULL) are automatically excluded by WHERE — SQL treats NULL as 'unknown', so NULL > 200 is unknown and gets filtered out.

Sample Output:

firm_id	fiscal_year	sales_m	net_income_m
103	2024	320	25
106	2023	260	16
106	2024	280	19
107	2023	310	21

107	2024	330	24

Query 9: Create Leverage = debt_m / assets_m

```
sqldf('SELECT firm_id, fiscal_year, debt_m, assets_m,
ROUND(debt_m * 1.0 / assets_m, 4) AS Leverage
FROM fundamentals')
```

Keywords Used: Arithmetic AS (alias) ROUND()

debt_m * 1.0 / assets_m — multiplying by 1.0 forces decimal division (without it, integer division might round incorrectly in some SQL engines).

AS Leverage — gives the calculated column a name. This is called an **alias**.

ROUND(x, 4) — rounds to 4 decimal places.

Leverage measures how much of assets are financed by debt. High leverage = more risk.

Sample Output:

firm_id	fiscal_year	debt_m	assets_m	Leverage
101	2023	190	520	0.3654
102	2024	430	850	0.5059
...	...	...	...	...

Query 10: List observations where sales_m IS NULL or net_income_m IS NULL

```
sqldf('SELECT * FROM fundamentals
WHERE sales_m IS NULL OR net_income_m IS NULL')
```

Keywords Used: IS NULL OR

IS NULL — the correct SQL way to check for missing values. Never use = NULL; it doesn't work in SQL!

OR — at least one condition must be true. Here: show the row if sales OR income is missing.

Three rows match: firm 103 in 2023 (sales NULL), firm 105 in 2024 (sales NULL), firm 105 in 2024 (net_income NULL too).

Sample Output:

firm_id	fiscal_year	sales_m	net_income_m
103	2023	NULL	18
105	2024	NULL	NULL

Note: *Always use IS NULL / IS NOT NULL, never = NULL or != NULL.*

Query 11: Count observations and average sales by fiscal year

```
sqldf('SELECT fiscal_year,
COUNT(*) AS N,
ROUND(AVG(sales_m), 2) AS AvgSales
FROM fundamentals
GROUP BY fiscal_year')
```

Keywords Used: COUNT() AVG() GROUP BY AS

GROUP BY fiscal_year — splits the table into groups (one per year), then applies aggregate functions to each group separately.

COUNT(*) — counts all rows in each group. Named N with AS.

AVG(sales_m) — calculates the average of sales_m within each group. AVG automatically ignores NULL values.

Aggregate functions (COUNT, AVG, SUM, MIN, MAX) always work together with GROUP BY.

Sample Output:

fiscal_year	N	AvgSales
2023	8	171.43
2024	8	223.57

Note: *AVG ignores NULLs automatically — only non-NULL sales_m values are averaged.*

TASK 2 PART B — Two-Table Queries

Understanding JOINs — The Most Important SQL Concept

A **JOIN** combines rows from two tables based on a matching key column.

We have firms (8 rows, one per firm) and fundamentals (16 rows, two years per firm).

INNER JOIN ... ON firms.firm_id = fundamentals.firm_id matches each firm row to its corresponding fundamentals rows. The result has 16 rows (each firm appears twice).

The ON clause is the 'bridge' — it tells SQL which columns must match.

Rule: always use a column that uniquely identifies entities (a key) in your ON clause.

Query 12: Join firms and fundamentals — return firm_name, sector, fiscal_year, sales_m

```
sqldf('SELECT f.firm_name, f.sector, fu.fiscal_year, fu.sales_m
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id')
```

Keywords Used: INNER JOIN ON Table Aliases

FROM firms f — 'f' is a short alias for 'firms'. Makes the query easier to read.

INNER JOIN fundamentals fu — join the fundamentals table (aliased as 'fu').

ON f.firm_id = fu.firm_id — the join condition: match rows where firm_id is equal.

INNER JOIN keeps only rows that have a match in BOTH tables. If a firm_id exists in firms but not in fundamentals, it is excluded.

Sample Output:

firm_name	sector	fiscal_year	sales_m
Alpha plc	Tech	2023	140
Alpha plc	Tech	2024	160
Beta Ltd	Manufacturing	2023	190
...	...	...	...

Query 13: After joining, return only Energy firms

```
sqldf("SELECT f.firm_name, fu.fiscal_year, fu.sales_m, fu.assets_m
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
WHERE f.sector = 'Energy'")
```

Keywords Used: JOIN + WHERE filter

We use the same JOIN as Q12, but add a WHERE clause to filter only Energy firms.

Key insight: **WHERE runs after the JOIN** — SQL first merges the tables, then filters rows. Think of it as: JOIN creates a big combined table, then WHERE trims it.

Energy firms: Cobalt SA (103), Fjord GmbH (106), Gamma Energy (107) — 6 rows total.

Sample Output:

firm_name	fiscal_year	sales_m	assets_m
Cobalt SA	2023	NULL	1520
Cobalt SA	2024	320	1550
Fjord GmbH	2023	260	1400
Gamma Energy	2024	330	1680

Query 14: Create ROA = net_income_m / assets_m, exclude NULL net incomes

```
sqldf('SELECT f.firm_name, fu.fiscal_year,
ROUND(fu.net_income_m * 1.0 / fu.assets_m, 4) AS ROA
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
WHERE fu.net_income_m IS NOT NULL')
```

Keywords Used: Calculated column + IS NOT NULL filter

IS NOT NULL — the opposite of IS NULL; keeps only rows where net income has a value.

ROA (Return on Assets) = net income / total assets. It measures how efficiently a firm uses its assets to generate profit.

This query excludes firm 105 in 2024 (net_income_m is NULL) — 15 rows remain.

Sample Output:

firm_name	fiscal_year	ROA
Alpha plc	2023	0.0423
Alpha plc	2024	0.0444
Beta Ltd	2023	0.0122
...	...	...

Query 15: Count firm-year observations by sector, sort descending

```
sqldf('SELECT f.sector, COUNT(*) AS N
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
GROUP BY f.sector
ORDER BY N DESC')
```

Keywords Used: GROUP BY COUNT(*) ORDER BY DESC

GROUP BY f.sector — after joining, group the 16 combined rows by sector.

COUNT(*) — count rows per sector group.

ORDER BY N DESC — sort results by N from highest to lowest. Use ASC for lowest to highest (ASC is the default if you omit it).

Energy has 3 firms × 2 years = 6 rows; Tech has 3 firms × 2 years = 6 rows; Manufacturing 2, Retail 2.

Sample Output:

sector	N
Energy	6
Tech	6
Manufacturing	2
Retail	2

Two-Table Queries Using 'firms' and 'market'

Query 16: Join firms and market — return firm_name, country, esg_score, market_cap_m

```
sqldf('SELECT f.firm_name, f.country, m.esg_score, m.market_cap_m
FROM firms f
INNER JOIN market m ON f.firm_id = m.firm_id')
```

Keywords Used: INNER JOIN (firms + market)

Same JOIN technique, different second table. firms and market both have one row per firm_id, so the result also has 8 rows (1:1 join).

Compare with Q12 (firms + fundamentals) which gave 16 rows (1:many join, since fundamentals has 2 years per firm).

Sample Output:

firm_name	country	esg_score	market_cap_m
Alpha plc	UK	74	1000
Beta Ltd	UK	56	1500
Cobalt SA	DE	50	2000
...	...	...	...

Query 17: Create ESG Group using CASE WHEN

```
sqldf("SELECT f.firm_name, m.esg_score,
CASE WHEN m.esg_score >= 75 THEN 'High'
WHEN m.esg_score >= 60 THEN 'Medium'
ELSE 'Low'
END AS ESG_Group
FROM firms f
INNER JOIN market m ON f.firm_id = m.firm_id")
```

Keywords Used: CASE WHEN THEN ELSE END

CASE WHEN is SQL's version of an if/else statement. It checks conditions top to bottom and returns the first THEN value where the condition is true.

ELSE is the default — if no WHEN condition matched, return 'Low'.

END closes the CASE block. Without END, SQL throws an error.

Order matters: SQL checks >= 75 first, then >= 60. If both were reversed, a score of 80 would incorrectly match the >= 60 group first.

Sample Output:

firm_name	esg_score	ESG_Group
Alpha plc	74	Medium
Beta Ltd	56	Low
Cobalt SA	50	Low
Epsilon Co	82	High
Helios Tech	79	High

TASK 2 PART C — Three-Table Queries

Chaining Multiple JOINs

To join three tables, simply chain two JOIN clauses one after the other.

SQL reads them left to right: first join firms with fundamentals, then join that result with market.

The join key for each step must match: here firm_id links all three tables.

Query 18: Join all three tables — return core fields

```
sqldf('SELECT f.firm_name, f.sector, fu.fiscal_year,
fu.sales_m, m.esg_score, m.market_cap_m
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
INNER JOIN market m ON f.firm_id = m.firm_id')
```

Keywords Used: Two INNER JOINs chained

First JOIN: firms + fundamentals (on firm_id) → 16 rows.

Second JOIN: (firms+fundamentals) + market (on firm_id) → still 16 rows, because market is 1 row per firm and gets duplicated for each year.

All three tables share firm_id as the common link — this is a **star schema** pattern common in finance databases.

Sample Output:

firm_name	sector	fiscal_year	sales_m	esg_score	market_cap_m
Alpha plc	Tech	2023	140	74	1000
Alpha plc	Tech	2024	160	74	1000
Beta Ltd	Manufacturing	2023	190	56	1500
...	...	...	...	...	...

Query 19: Create Leverage and ROA in a three-table join

```
sqldf('SELECT f.firm_name, fu.fiscal_year,
ROUND(fu.debt_m * 1.0 / fu.assets_m, 4) AS Leverage,
ROUND(fu.net_income_m * 1.0 / fu.assets_m, 4) AS ROA,
m.esg_score
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
INNER JOIN market m ON f.firm_id = m.firm_id')
```

Keywords Used: Multiple calculated columns in a 3-table JOIN

We combine multiple computed columns (Leverage and ROA) in a single SELECT.

Each calculation uses only fundamentals columns — we just need firms and market joined in order to access their name and ESG score respectively.

Rows with NULL net_income_m will have NULL ROA — that's fine here since the question doesn't require filtering them out.

Sample Output:

firm_name	fiscal_year	Leverage	ROA	esg_score
Alpha plc	2023	0.3654	0.0423	74
Epsilon Co	2024	0.3333	NULL	82
...	...	...	...	...

Query 20: Average leverage by sector — keep sectors with >= 4 observations

```
sqldf('SELECT f.sector,
COUNT(*) AS N,
ROUND(AVG(fu.debt_m * 1.0 / fu.assets_m), 4) AS AvgLeverage
FROM firms f
INNER JOIN fundamentals fu ON f.firm_id = fu.firm_id
INNER JOIN market m ON f.firm_id = m.firm_id
GROUP BY f.sector
HAVING COUNT(*) >= 4')
```

Keywords Used: GROUP BY HAVING AVG on a calculated expression

HAVING is like WHERE but it filters GROUPS, not individual rows.

Rule: use WHERE to filter rows before grouping; use HAVING to filter groups after grouping.

HAVING COUNT(*) >= 4 — only keep sectors where at least 4 firm-year rows exist.

Energy (6 obs) and Tech (6 obs) pass. Manufacturing (2) and Retail (2) are excluded.

Sample Output:

sector	N	AvgLeverage
Energy	6	0.6196
Tech	6	0.4142

Note: *HAVING always comes after GROUP BY and before ORDER BY in the query structure.*

TASK 2 PART D — Subqueries and CTEs

What is a Subquery?

A **subquery** is a SELECT statement nested inside another SELECT statement.

Think of it as: first run the inner query to get a temporary result, then use that result in the outer query.

Subqueries can appear in the FROM clause (inline view), WHERE clause, or SELECT list.

Query 21: Subquery: firms with market_cap_m >= 1200, then join to fundamentals

```
sqldf('SELECT sub.firm_name, fu.fiscal_year, fu.sales_m, sub.market_cap_m
FROM (
SELECT f.firm_id, f.firm_name, m.market_cap_m
FROM firms f
INNER JOIN market m ON f.firm_id = m.firm_id
WHERE m.market_cap_m >= 1200
) AS sub
INNER JOIN fundamentals fu ON sub.firm_id = fu.firm_id')
```

Keywords Used: Subquery in FROM clause (Inline View)

The inner query (inside parentheses) finds firms with large market caps and gives them alias 'sub'.

AS sub — we must give the subquery an alias so the outer query can reference it.

The outer query then joins this filtered set with fundamentals.

Firms with market_cap_m >= 1200: Beta Ltd (1500), Cobalt SA (2000), Fjord GmbH (1800), Gamma Energy (2200), Helios Tech (1200) — 5 firms × 2 years = 10 rows.

Sample Output:

firm_name	fiscal_year	sales_m	market_cap_m
Beta Ltd	2023	190	1500
Beta Ltd	2024	210	1500
Cobalt SA	2023	NULL	2000
Gamma Energy	2024	330	2200
...	...	...	...

Query 22: Subquery in WHERE: firm-years where leverage > average leverage

```
sqldf('SELECT firm_id, fiscal_year,
ROUND(debt_m * 1.0 / assets_m, 4) AS Leverage
FROM fundamentals
WHERE (debt_m * 1.0 / assets_m) > (
SELECT AVG(debt_m * 1.0 / assets_m) FROM fundamentals
)')
```

Keywords Used: Subquery in WHERE clause (scalar subquery)

The inner query **SELECT AVG(debt_m * 1.0 / assets_m) FROM fundamentals** calculates a single number — the overall average leverage across all 16 rows. This is called a **scalar subquery** (returns one value).

The outer query compares each row's leverage to that single average number.

Full-sample average leverage ≈ 0.609. Rows above this are high-leverage firms.

Sample Output:

firm_id	fiscal_year	Leverage
103	2023	0.6053
103	2024	0.6129
106	2023	0.5571
107	2023	0.6125
107	2024	0.5952
...	...	...

Query 23: CTE: UK firms only, then join to fundamentals and market

```
sqldf('WITH uk_firms AS (
SELECT firm_id, firm_name
FROM firms
WHERE country = \'UK\'
)
SELECT uk.firm_name, fu.fiscal_year, fu.sales_m, m.esg_score
FROM uk_firms uk
INNER JOIN fundamentals fu ON uk.firm_id = fu.firm_id
INNER JOIN market m ON uk.firm_id = m.firm_id')
```

Keywords Used: CTE (WITH clause)

CTE = Common Table Expression. A CTE creates a named temporary result set that you can reference in the main query — like giving a subquery a permanent name.

WITH uk_firms AS (...) — defines the CTE named 'uk_firms'.

The main query then treats uk_firms like a normal table.

CTEs make complex queries much more readable than nested subqueries. They are especially useful when you need to reuse the same intermediate result multiple times.

UK firms: Alpha plc (101), Beta Ltd (102), Epsilon Co (105), Gamma Energy (107) — 4 firms × 2 years = 8 rows.

Sample Output:

firm_name	fiscal_year	sales_m	esg_score
Alpha plc	2023	140	74
Alpha plc	2024	160	74
Beta Ltd	2023	190	56
Gamma Energy	2024	330	68
...	...	...	...

TASK 3 — Sample Short Summary Report (500 words)

Structure of the Three Tables and Their Links

This assignment uses three related tables. The **firms** table contains one row per company, recording static characteristics such as sector, country, and founding year. The **fundamentals** table is organised at the firm-year level, meaning each firm appears twice — once for 2023 and once for 2024 — with accounting variables including sales, assets, debt, and net income. The **market** table holds one row per firm, capturing market-based indicators such as ESG score, market capitalisation, and the stock exchange listing. All three tables are linked through the shared key **firm_id**, enabling JOIN operations to combine information across sources.

Main Results from Filtering and Grouping Queries

The one-table queries revealed that the sample spans four sectors (Tech, Manufacturing, Energy, Retail) and four countries (UK, DE, FR, US). Filtering on fundamentals showed that five firm-year observations recorded sales above 200 million, all belonging to Energy firms. When leverage was computed as debt-to-assets and grouped by fiscal year, no meaningful year-on-year change was detected, suggesting broadly stable capital structures across the sample.

Join Query Results

After joining firms and fundamentals, the Energy sector and Tech sector each accounted for six firm-year observations, making them the largest groups. ROA values were generally low, with Energy firms showing relatively higher profitability due to large asset bases. The ESG classification query revealed that two firms (Epsilon Co and Helios Tech) achieved a High ESG rating (score >= 75), while three firms were classified as Low, indicating room for improvement across the sample. Sectors with at least four observations had average leverage of approximately 0.62 (Energy) and 0.41 (Tech), suggesting Energy firms are more heavily debt-financed.

Missing Values and Data Issues

Two missing values were identified in the fundamentals table: firm 103 (Cobalt SA) had no sales figure for 2023, and firm 105 (Epsilon Co) had neither sales nor net income recorded for 2024. These NULLs required careful handling. The IS NULL filter in Query 10 successfully isolated these observations. Importantly, aggregate functions such as AVG() automatically excluded NULLs from calculations, which preserved result accuracy without requiring manual removal.

Reflection on SQL for Corporate Finance Analysis

SQL provides an efficient framework for corporate finance data work. Rather than manually linking spreadsheets, JOIN operations automatically combine firm characteristics with accounting and market data in a single query. GROUP BY with aggregate functions enables rapid summary statistics by sector or year. Subqueries and CTEs allow multi-step filtering — for example, isolating high-market-cap firms before analysing their fundamentals — without creating intermediate datasets. Overall, SQL reduces the risk of manual errors and makes the analytical workflow transparent and reproducible.

Quick Reference: SQL Keywords Covered

Keyword / Clause	What it does	Example
SELECT *	Return all columns	`SELECT * FROM firms`
SELECT col1, col2	Return specific columns	`SELECT firm_name, sector FROM firms`
DISTINCT	Remove duplicate rows	`SELECT DISTINCT sector FROM firms`
WHERE	Filter rows by condition	`WHERE country = 'UK'`
AND / OR	Combine conditions	`WHERE year > 2005 AND country != 'DE'`
LIKE '%x%'	Pattern match on text	`WHERE firm_name LIKE '%Tech%'`
IS NULL / IS NOT NULL	Check for missing values	`WHERE sales_m IS NULL`
AS	Rename a column (alias)	`debt_m/assets_m AS Leverage`
ORDER BY ... DESC	Sort results	`ORDER BY N DESC`
GROUP BY	Group rows for aggregation	`GROUP BY sector`
HAVING	Filter groups	`HAVING COUNT(*) >= 4`
COUNT(*)	Count rows	`COUNT(*) AS N`
AVG()	Average (ignores NULLs)	`AVG(sales_m) AS AvgSales`
INNER JOIN ... ON	Combine two tables on a key	`JOIN fundamentals fu ON f.firm_id = fu.firm`
CASE WHEN...THEN...ELSE...END	Conditional column	`CASE WHEN esg >= 75 THEN 'High' END`
Subquery (FROM)	Use a query result as a table	`FROM (SELECT ... WHERE ...) AS sub`
Subquery (WHERE)	Compare against a query result	`WHERE leverage > (SELECT AVG(leverage)...)`
WITH ... AS (CTE)	Named temporary result set	`WITH uk_firms AS (SELECT ... WHERE ...)`
ROUND(x, n)	Round to n decimal places	`ROUND(debt_m/assets_m, 4)`